

Module 17: Django REST Framework

1. Introduction to APIs

1 What is an API (Application Programming Interface)?

An API (Application Programming Interface) is a set of rules and protocols that allows different

software applications to communicate with each other. It acts as an intermediary, enabling one system to request data or functionality from another.

In simple words:

An API is like a waiter in a restaurant. You (the client) give your order to the waiter (API), who takes it to the kitchen (server/database), and then brings your food (response) back to you.

Example:

Frontend (React) wants user data → Calls API (`/api/users/``) → API fetches data from database → API returns JSON data to Frontend.

2 Types of APIs: REST, SOAP

1 REST (Representational State Transfer)

REST is an architectural style that uses standard HTTP methods (GET, POST, PUT, DELETE). It

usually exchanges data in JSON format.

- It is stateless (server does not store client session)
- It is lightweight, fast, and highly scalable
- Easy to use with modern web and mobile apps

2 SOAP (Simple Object Access Protocol)

SOAP is a strict protocol that uses XML for exchanging structured information.

- It is highly standardized
- Supports advanced security (WS -Security)
- Mostly used in banking and enterprise -level applications

Summary of API Types

Type	Format	Use Case
REST	JSON/XML	Modern web and mobile apps
SOAP	XML	High-security enterprise apps

3 Why are APIs important in web development?

- Decoupling: Frontend and backend can be developed independently
- Reusability: Same API can be used for Web, Android, and iOS apps
- Third-Party Integration: Allows integrating services like Stripe (payments) without building them from scratch

2. Requirements for Web Development Projects

1 Understanding project requirements.

Before coding, developers must analyze specific needs. This involves:

- Identifying target audience and project goals
- Determining required features (Authentication, Payment, etc.)
- Designing database schema
- Defining API endpoints

2 Setting up the environment and installing necessary packages.

It is highly recommended to use a virtual environment to isolate project dependencies.

Steps to setup:

1 Create virtual environment:

```
python -m venv venv
```

2 Activate virtual environment:

```
venv\Scripts\activate (on Windows)
```

3 Install required packages:

```
pip install django djangorestframework
```

3. Serialization in Django REST Framework (DRF)

1 What is Serialization?

Serialization is the process of converting complex data types (like Django models or QuerySets) into a format that can be easily transferred over the network, such as JSON or XML.

In simple words:

Serialization means converting Python objects into JSON format so that APIs can send data to

frontend or other systems.

Why serialization is needed:

- Web browsers and mobile apps understand JSON, not Python objects
- APIs need data in a standard format
- Helps in communication between backend and frontend

Example:

Python object (Django model instance):

User object → name, email, id

After serialization:

```
{  
  "id": 1,  
  "name": "Bharti",  
  "email": "bharti@example.com"  
}
```

2 Converting Django QuerySets to JSON

A QuerySet in Django is a collection of database records.

Example:

```
users = User.objects.all()
```

This QuerySet cannot be directly sent to the frontend. It must be converted into JSON format.

Steps to convert QuerySet to JSON:

Method 1 Using Django serializers (built-in)

```
from django.core import serializers
```

```
data = serializers.serialize("json", User.objects.all())
```

This converts QuerySet into JSON format.

Example output:

```
[  
  {  
    "model": "app.user",  
    "pk": 1,  
    "fields": {  
      "name": "Bharti",
```

```
"email": "bharti@example.com"
}
}
]
```

Method 2 Using DRF Serializers (recommended)

DRF serializers convert QuerySets into clean JSON format.

Example:

```
from rest_framework import serializers
```

```
class UserSerializer(serializers.ModelSerializer):
```

```
    class Meta:
```

```
        model = User
```

```
        fields = '__all__'
```

Then:

```
serializer = UserSerializer(User.objects.all(), many=True)
```

```
return Response(serializer.data)
```

Output:

```
[
{
    "id": 1,
    "name": "Bharti",
```

```
"email": "bharti@example.com"
}
]
```

3 Using Serializers in Django REST Framework (DRF)

Serializers in DRF are used to convert complex data (like models) into JSON and also validate incoming data.

Types of Serializers:

1 ModelSerializer

Automatically creates serializer based on Django model.

Example:

```
from rest_framework import serializers
from .models import User
```

```
class UserSerializer(serializers.ModelSerializer):
    class Meta:
        model = User
        fields = '__all__'
```

Advantages:

- Less code

- Easy to use
- Automatically maps model fields

2 Serializer (manual)

Used when you need full control over fields.

Example:

```
class UserSerializer(serializers.Serializer):  
    name = serializers.CharField(max_length=100)  
    email = serializers.EmailField()
```

4. HTTP Request Methods and DRF Responses

1 HTTP Request Methods (GET, POST, PUT, DELETE)

HTTP request methods define what action the client wants to perform on the server. In REST API and Django REST Framework (DRF), these methods are used to perform CRUD operations.

1 GET Method

GET method is used to retrieve data from the server.

- It does not change data on the server
- It is a read -only operation
- Data is usually sent in the URL

Example:

GET /api/users/

Use case:

- Fetch all users
- Get product list

Example response:

```
[  
{  
  "id": 1,  
  "name": "Bharti"  
}  
]
```

2 POST Method

POST method is used to send data to the server and create a new resource.

- It adds new data into the database
- Data is sent in the request body
- Commonly used for registration

Example:

POST /api/users/

Request body:

```
{  
  "name": "Bharti",  
  "email": "bharti@example.com"  
}
```

Use case:

- User registration
- Creating new product

3 PUT Method

PUT method is used to update an existing resource completely.

- It replaces the entire object

Example:

PUT /api/users/1/

Request body:

```
{  
  "name": "Bharti Chavda",  
  "email": "bharti@gmail.com"  
}
```

Use case:

- Update full user profile

4 DELETE Method

DELETE method is used to remove a resource from the server.

- It deletes data permanently

Example:

DELETE /api/users/1/

Use case:

- Delete user account

Summary of HTTP Methods

Method	Action	Purpose
GET	Read	Fetch data
POST	Create	Add new data
PUT	Update	Replace existing data
DELETE	Delete	Remove data

2 Sending and Receiving Responses in DRF

In Django REST Framework, responses are handled using the Response class, which returns

data in JSON format.

1 Sending Response in DRF

DRF uses the Response object to send data back to the client.

Example:

```
from rest_framework.response import Response
```

```
def get_users(request):
```

```
    users = User.objects.all()
```

```
    serializer = UserSerializer(users, many=True)
```

```
    return Response(serializer.data)
```

What happens here:

- Data is fetched from database
- Serializer converts it into JSON format
- Response sends JSON to client

2 Receiving Request in DRF

DRF receives data from client through request object.

Example:

```
def create_user(request):
```

```
    data = request.data
```

```
    serializer = UserSerializer(data=data)
```

```
    if serializer.is_valid():
```

```
        serializer.save()
```

```
    return Response(serializer.data)
```

```
else:
```

```
return Response(serializer.errors)
```

Explanation:

- request.data contains incoming JSON data
- Serializer validates the data
- If valid, data is saved in database

3 Types of DRF Responses

a) Success Response

Used when operation is successful.

Example:

```
return Response({"message": "User created successfully"}, status=201)
```

b) Error Response

Used when validation fails or error occurs.

Example:

```
return Response(serializer.errors, status=400)
```

4 Status Codes in DRF

Status Code Meaning

200 OK (Success)

201 Created

400 Bad Request

404 Not Found

500 Server Error

5. Views in Django REST Framework

1 Understanding views in DRF: Function -based views vs Class -based views.

Views handle the logic of receiving a request and returning a response.

1 Function -Based Views (FBV)

FBVs use the `@api_view` decorator. They are easy to write for simple logic.

Example:

```
from rest_framework.decorators import api_view
```

```
from rest_framework.response import Response
```

```
@api_view(['GET'])
```

```
def get_books(request):
```

```
    return Response({"books": ["Book 1", "Book 2"]})
```

Use case:

- Simple endpoints
- Custom logic that doesn't fit standard database CRUD operations

2 Class-Based Views (CBV)

CBVs inherit from `APIView`. They are object -oriented and much better for complex logic and

reusability.

Example:

```
from rest_framework.views import APIView  
from rest_framework.response import Response
```

```
class BookView(APIView):  
  
    def get(self, request):  
        return Response({"books": ["Book 1", "Book 2"]})  
  
    def post(self, request):  
        return Response({"message": "Book added"})
```

Use case:

- Complex CRUD operations
- Code reusability
- Utilizing DRF's generic views

6. URL Routing in Django REST Framework

1 Defining URLs and linking them to views.

URL routing maps an API endpoint to a specific view.

1 Standard Routing

For Function -Based Views or simple Class -Based Views, you map them directly in urls.py.

Example:

```
from django.urls import path
```

```
from . import views
```

```
urlpatterns = [
```

```
path('books/', views.BookView.as_view(), name='book -list'),
```

```
]
```

2 Router-Based Routing

For ViewSets, DRF provides Routers which automatically generate all standard RESTful URL patterns.

Example:

```
from rest_framework.routers import DefaultRouter
```

```
from .views import UserViewSet
```

```
router = DefaultRouter()
```

```
router.register('users', UserViewSet)
```

```
urlpatterns = router.urls
```

7. Pagination in Django REST Framework

1 Adding pagination to APIs to handle large data sets.

Pagination splits large datasets into smaller chunks (pages).

In simple words:

Instead of sending 1000 records at once (which slows down the app), pagination sends 10 records per page.

1 PageNumberPagination

The client requests a specific page number.

Example Request:

GET /api/users/?page=2

Example Response:

```
{  
  "count": 1000,  
  "next": "http://api/users/?page=3",  
  "previous": "http://api/users/?page=1",  
  "results": [  
    { "name": "Bharti" },  
    ... 9 more records  
  ]  
}
```

Use case:

- Displaying data tables
- Blogs and articles lists

8. Settings Configuration in Django

1 Configuring Django settings for database, static files, and API keys.

The settings.py file controls the entire Django project.

1 Database Configuration

Defines where data is stored.

Example:

```
DATABASES = {  
  
    'default': {  
  
        'ENGINE': 'django.db.backends.sqlite3',  
  
        'NAME': BASE_DIR / 'db.sqlite3',  
  
    }  
}
```

2 Static Files Configuration

Defines where CSS, JS, and Images are stored.

Example:

```
STATIC_URL = '/static/'  
  
STATIC_ROOT = BASE_DIR / 'staticfiles'
```

3 API Keys

Never hardcode sensitive keys. Use environment variables.

Example:

```
import os
```

```
STRIPE_API_KEY = os.getenv("STRIPE_API_KEY")
```

9. Project Setup

1 Setting up a Django REST Framework project.

Step-by-step commands:

1 Create Project:

```
django-admin startproject myproject .
```

2 Create App:

```
python manage.py startapp myapp
```

3 Configure settings.py:

```
INSTALLED_APPS = [
```

```
...
```

```
'rest_framework',
```

```
'myapp',
```

```
]
```

4 Run migrations:

`python manage.py migrate`

5 Run server:

`python manage.py runserver`

10. Social Authentication, Email, and OTP Sending API

1 Implementing social authentication in Django.

Social auth allows users to login without creating a new password.

- Implemented using third -party packages like django -allauth.
- Eliminates the need to remember passwords.

Use case:

- "Login with Google" button.

2 Sending emails and OTPs using third -party APIs.

1 Email (SendGrid):

Used for transactional emails like password resets.

Example setup: Use SendGrid API key and Django's `send_mail` function.

2 SMS OTP (Twilio):

Used for sending text messages for 2 -factor authentication.

Example:

```
from twilio.rest import Client

client = Client('ACCOUNT_SID', 'AUTH_TOKEN')

message = client.messages.create(

    body="Your OTP is 1234",

    from_='+1234567890',

    to='+0987654321'

)
```

11. RESTful API Design

1 REST principles: statelessness, resource -based URLs, and HTTP methods.

1 Statelessness

Every API request must contain all info (like an Auth token). The server does not remember previous requests.

2 Resource -based URLs

URLs should be nouns (resources), not verbs (actions).

- Good: /api/products/

- Bad: /api/get_products/

3 HTTP methods for CRUD

Use proper verbs.

- GET /api/products/ (to read)
- POST /api/products/ (to create)

12. CRUD API (Create, Read, Update, Delete)

1 What is CRUD, and why is it fundamental to backend development?

CRUD stands for Create, Read, Update, and Delete. These are the 4 basic database operations.

In simple words:

Any app you use is basically doing CRUD operations on a database.

Operations:

- Create (POST) = Register account
- Read (GET) = View profile
- Update (PUT/PATCH) = Edit profile
- Delete (DELETE) = Delete account

13. Authentication and Authorization API

1 Difference between authentication and authorization.

Authentication: Verifying WHO you are.

- Example: Logging in with a username and password .

Authorization: Verifying WHAT you can do.

- Example: Only admin users can delete other users.

2 Implementing authentication using DRF token -based system.

In DRF, you can use TokenAuthentication.

When a user logs in, they receive a unique Token string.

They must send this token in the header of all future requests:

Example Header:

Authorization: Token 9944b09199c62bcf9418ad846dd0e4bbdfc6ee4b

14. OpenWeatherMap API Integration

1 Introduction to OpenWeatherMap API and how to retrieve weather data.

OpenWeatherMap provides real -time weather data for any location.

Example:

```
import requests
```

```
api_key = "YOUR_API_KEY"
```

```
city = "Ahmedabad"
```

```
url = f"http://api.openweathermap.org/data/2.5/weather?q={city}&appid={api_key}"
```

```
response = requests.get(url)

data = response.json()

print("Temperature:", data['main']['temp'])
```

15. Google Maps Geocoding API

1 Using Google Maps Geocoding API to convert addresses into coordinates.

Geocoding means converting an address string into Latitude and Longitude coordinates.

In simple words:

You give the API "Navrangpura, Ahmedabad", and it gives you back coordinates (Lat: 23.0366, Lng: 72.5611).

Use case:

- Pinning locations on a map
- Calculating distances

16. GitHub API Integration

1 Introduction to GitHub API and how to interact with repositories.

GitHub provides a REST API that lets developers automate tasks on the platform.

Using a Personal Access Token, you can write a Python script to:

- Automatically create a new repository
- List all open pull requests
- Add comments to issues

17. Twitter API Integration

1 Using Twitter API to fetch and post tweets, and retrieve user data.

Using the tweepy Python library, you can authenticate your app with Twitter.

Use case:

- Programmatically posting new tweets
- Reading the latest tweets containing a specific hashtag
- Building a social media dashboard

18. REST Countries API Integration

1 Introduction to REST Countries API and how to retrieve country -specific data.

REST Countries is a free public API providing country data without needing an API key.

Example Request:

GET <https://restcountries.com/v3.1/name/india>

Example Response:

Returns a JSON object containing India's population, borders, currencies, languages, and flag

image.

19. Email Sending APIs (SendGrid, Mailchimp)

1 Using email sending APIs like SendGrid and Mailchimp to send transactional emails.

Why use an API instead of Django's default?

Instead of sending emails from a local server (which usually go straight to the Spam folder), developers use professional services like SendGrid.

Use case:

- Sending welcome emails
- Sending password reset links
- Tracking if the email was opened

20. SMS Sending APIs (Twilio)

1 Introduction to Twilio API for sending SMS and OTPs.

Twilio is a cloud communications platform that allows developers to send SMS texts globally.

Use case:

- Sending OTPs for Two -Factor Authentication
- Sending automated appointment reminders

21. Payment Integration (PayPal, Stripe)

1 Introduction to integrating payment gateways like PayPal and Stripe.

Payment gateways securely process credit card transactions without storing card details on

your own server.

How it works:

- 1. Frontend shows a secure card form, and generates a Token.**
- 2. Backend (Django) receives the Token.**
- 3. Backend calls the Stripe/PayPal API with a Secret Key, and charges the customer.**

22. Google Maps API Integration

1 Using Google Maps API to display maps and calculate distances between locations.

Besides Geocoding, Google Cloud provides several location APIs:

- Maps JavaScript API: To embed and show interactive maps on a website.
- Distance Matrix API: To calculate the exact travel time and distance between an origin and destination, based on real-time traffic.

Use case:

- Used by apps like Uber and Zomato to estimate delivery times.